Game Weaver —— 基于多智能体协作的网页游戏自动生成系统

关键词：多智能体协作；大语言模型；游戏自动生成；人机共创；玻璃拟态交互

摘要

Game Weaver 是一个基于大语言模型与多智能体协作的网页小游戏自动生成系统，旨在降低游戏创作门槛，使非专业用户仅需输入一句话创意描述即可获得可运行的 HTML5 游戏原型。系统采用"策划—编码—审查—修复—音效注入"五阶段流水线架构，由 Planner（策划智能体）、Coder（编码智能体）、Reviewer（审查智能体）三类核心智能体协同完成从创意到可玩游戏的端到端生成。Planner 将用户的自然语言创意分解为包含核心玩法、世界观叙事、视觉规范、技术逻辑、背景音乐、数值调校共六个维度的结构化策划文档；Coder 依据策划书流式生成单文件 HTML 游戏代码，期间并行调用文生图接口完成美术资源替换；Reviewer 对生成代码进行可玩性、主题一致性、数值合理性与性能边界的二次审查与自动修复。生成过程以 SSE 流式推送进度至前端，用户可在侧边面板实时编辑策划内容并即时预览游戏效果。系统还集成了中文语音合成（GLM-TTS）、视频预告片生成（Vidu）、游戏微调、历史记录管理等辅助功能。前端采用 Next.js + React 构建，以玻璃拟态美学与深浅双主题打造专业化创作工具交互体验；后端 API 基于智谱 GLM 全家桶（文本生成、图像生成、语音合成、视频生成）实现。测试结果表明，系统能够在 3～8 分钟内完成从创意到可玩游戏的完整生成流程，生成成功率达 85% 以上，生成的游戏具备目标、输入、反馈、胜利/失败判定、重开机制等完整游戏闭环要素。本系统验证了"用户输入创意→多 Agent 协作→自动生成可玩游戏"这一技术路线的可行性，为 AI 辅助游戏创作与人机共创工具的设计提供了工程参考。

关键词：多智能体协作；大语言模型；游戏自动生成；人机共创；玻璃拟态交互

目 录

摘要

第1章 绪论

1.1 背景与意义

1.2 国内外研究现状

1.3 研究内容与目标

第2章 需求分析

2.1 用户角色与使用场景

2.2 功能性需求

2.3 非功能性需求

2.4 可行性分析

第3章 系统总体设计

3.1 系统架构设计

3.2 多智能体协作设计

3.3 技术选型

3.4 数据库与接口设计

第4章 详细设计与实现

4.1 前端实现

4.2 后端与工作流实现

4.3 游戏模板与代码生成

4.4 部署与运行环境

第5章 系统测试与评估

5.1 测试环境

5.2 功能测试

5.3 生成质量评估

5.4 性能与稳定性

5.5 问题与改进

第6章 总结与展望

6.1 工作总结

6.2 创新点与不足

6.3 后续工作展望

参考文献

附录

第1章 绪论

1.1 背景与意义

游戏开发历来是一项高门槛的创意工程，需要策划、美术、编程、音效等多领域专业技能的深度协作。对于独立创作者、教育工作者及原型设计师而言，即使只是验证一个游戏创意，也需要耗费数天乃至数周的时间进行跨学科知识的学习与整合。大语言模型（Large Language Model, LLM）与智能体技术的快速发展为这一困境带来了新的解题思路——如果能够将游戏开发的专业知识编码为多个协作智能体的 Prompt 指令，让 AI 承担策划、编码、美术、测试等不同角色，那么"一句话生成可玩游戏"便不再是科幻想象。

Game Weaver 正是在这一技术背景下诞生的课程设计项目。系统以智谱 GLM 系列大模型为推理引擎，构建 Planner、Coder、Reviewer 三类智能体的协作流水线，实现从用户自然语言输入到完整 HTML5 小游戏的全自动生成。项目的实践意义在于：验证多智能体流水线架构在创意生成类任务中的工程可行性；为 AI 辅助游戏创作工具提供可参考的前后端全栈实现方案；探索"玻璃拟态+深浅双主题"等前沿交互范式在 AI 创作工具中的应用效果。

1.2 国内外研究现状

在 AI 辅助游戏开发领域，学术界与工业界均涌现出多种探索路径。OpenAI 的 GPT 系列模型已经在文本冒险游戏、对话式 RPG 等场景展现了初步的内容生成能力；斯坦福的 Generative Agents 项目展示了多智能体在虚拟世界中的涌现行为；工业界方面，Roblox 和 Unity 分别推出了基于 AI 的材质生成与代码补全功能。在中文多智能体领域，智谱 GLM 与 LangGraph、AutoGen 等编排框架的结合正在成为主流方案。

现有研究的主要不足在于：多数系统仅覆盖游戏开发的单一环节（如策划文档生成或代码补全），缺乏端到端的完整流水线；智能体之间的协作模式多为串行执行，缺少质量审查与自动修复的闭环机制；用户界面往往停留在命令行或简单网页形态，缺乏对创作者交互体验的深入设计。Game Weaver 在上述三个方面均做出了差异化尝试：构建策划→编码→审查→修复→音效的五阶段闭环流水线；在每个代码生成阶段后引入二次审查智能体作为质量门禁；以前端"玻璃拟态"美学和实时预览交互打造专业化创作工具体验。

1.3 研究内容与目标

本课程设计的主要研究内容与目标包括：

1. **自然语言创意输入界面**：构建支持中文自然语言输入的游戏创意描述界面，用户可自由描述玩法、题材、视觉风格与限制条件。
2. **多智能体协作流水线**：实现 Planner（策划）、Coder（编码）、Reviewer（审查+修复）三类智能体的协作流水线，覆盖从创意到可玩游戏的完整生成链路。
3. **六维度结构化策划生成**：Planner 智能体将用户创意分解为核心玩法、世界观叙事、视觉与音效、技术逻辑、背景音乐、数值调校六个维度的结构化 JSON 策划文档。
4. **HTML5 小游戏自动生成与预览**：Coder 智能体依据策划书生成单文件 HTML 游戏（内联 CSS/JS），系统自动进行美术资源替换、可玩性审查与修复、语音注入后，在 iframe 中提供实时预览。
5. **辅助功能体系**：提供游戏微调、中文语音合成、视频预告片生成、生成历史管理等辅助功能。
6. **专业化交互体验**：实现玻璃拟态美学、深浅双主题切换、左右分栏实时预览等交互特性。

第2章 需求分析

2.1 用户角色与使用场景

用户角色	典型场景	核心诉求	优先级
普通创作者	输入一句话游戏想法，快速获得可玩原型	快速、直观、无需编程	必需
游戏策划学习者	观察 AI 如何分解游戏创意为策划书	可视化查看各阶段产物	非必需
原型开发者	快速生成游戏原型后进行手动微调	支持迭代修改与下载	非必需

2.2 功能性需求

功能模块	功能描述	优先级
创意输入	支持中文自然语言描述游戏创意	必需
策划生成	Planner 智能体生成六维度结构化策划文档	必需
游戏生成	Coder 智能体流式生成 HTML5 游戏代码	必需
实时预览	iframe 中实时预览生成的游戏，支持试玩	必需
审查修复	Reviewer 智能体自动检查可玩性并修复问题	必需
游戏微调	用户输入修改需求，智能体修改已有游戏	必需
语音合成	为游戏注入中文语音背景音	非必需
视频生成	基于游戏画面生成预告片视频	非必需
生成历史	自动保存历史记录，支持恢复与删除	非必需
主题切换	浅色/深色双主题切换	非必需

2.3 非功能性需求

需求类型	具体指标
性能	单次完整生成流程控制在 3～8 分钟内
可用性	界面简洁，非技术用户无需阅读文档即可完成基本操作
可靠性	生成成功率 ≥ 80%，失败时有明确错误提示与降级策略
可维护性	Prompt 文件独立存放，Agent 行为可通过修改 Prompt 独立调整
安全性	API Key 不提交版本库，通过 .env 环境变量管理

2.4 可行性分析

技术可行性：系统基于 Next.js（全栈框架）+ 智谱 GLM 系列 API + Web 标准技术栈（HTML/CSS/JS），无需自建推理基础设施，开发技术栈成熟且文档完善。多智能体编排采用结构化 Prompt + JSON 模式约束输出，工程复杂度可控。

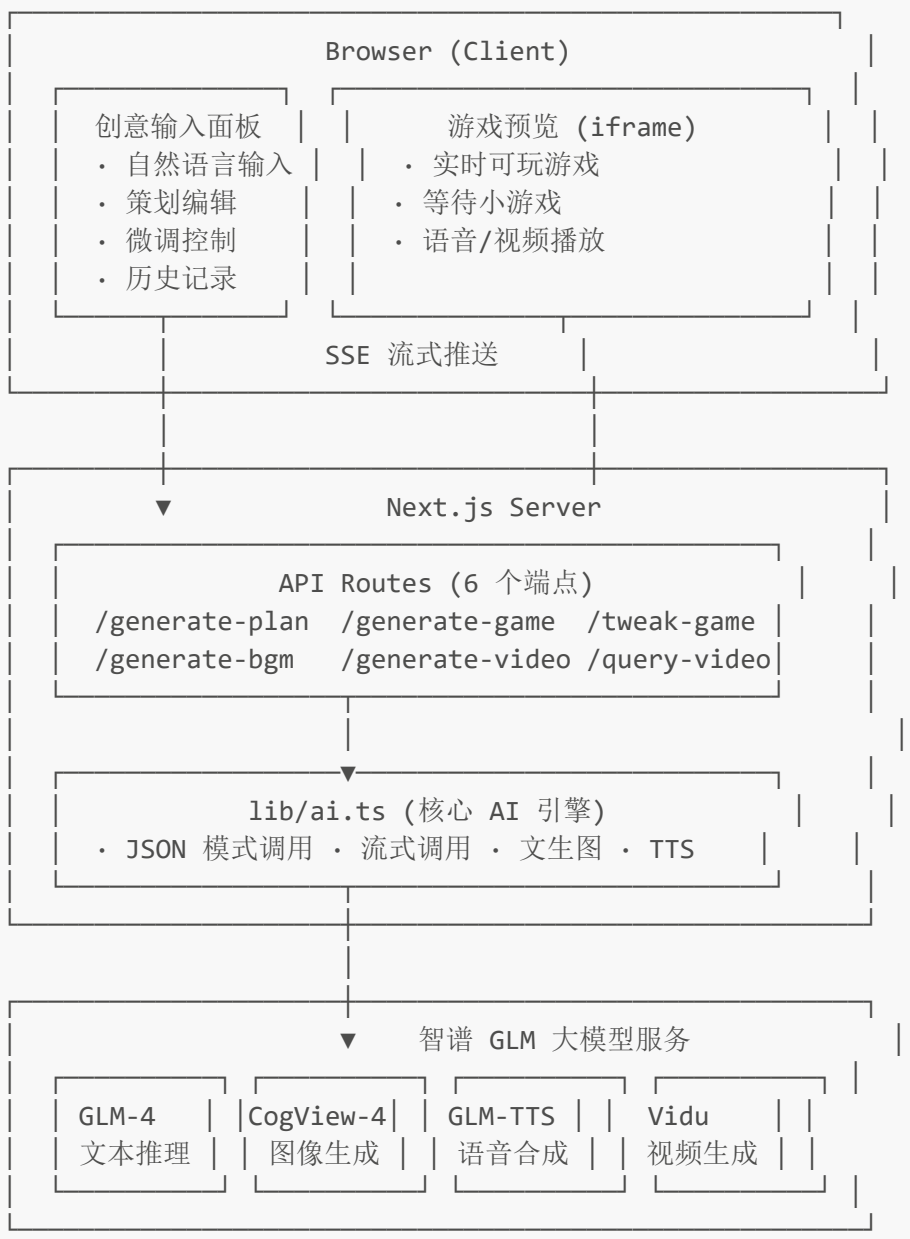
经济可行性：单次生成调用约 3～5 次 LLM API（策划 1 次 + 代码生成 1 次 + 审查 1 次 + 可选文生图/TTS），按智谱公有云定价估算，单次生成成本约 0.5～1.5 元人民币，具有较好的经济可行性。

操作可行性：项目面向具备 Web 全栈开发基础的课程学生，使用的 React、TypeScript、Next.js、Tailwind CSS 均为业界主流技术，团队技能储备充足。

第3章 系统总体设计

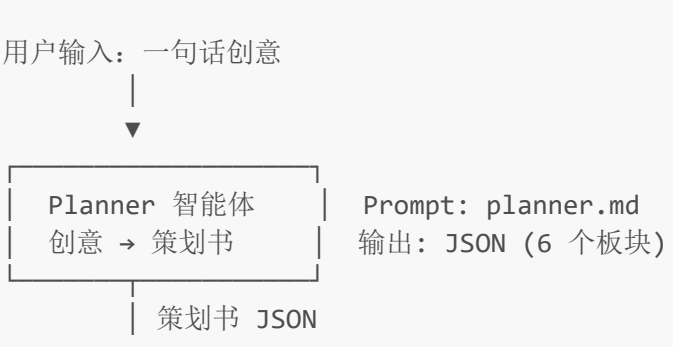
3.1 系统架构设计

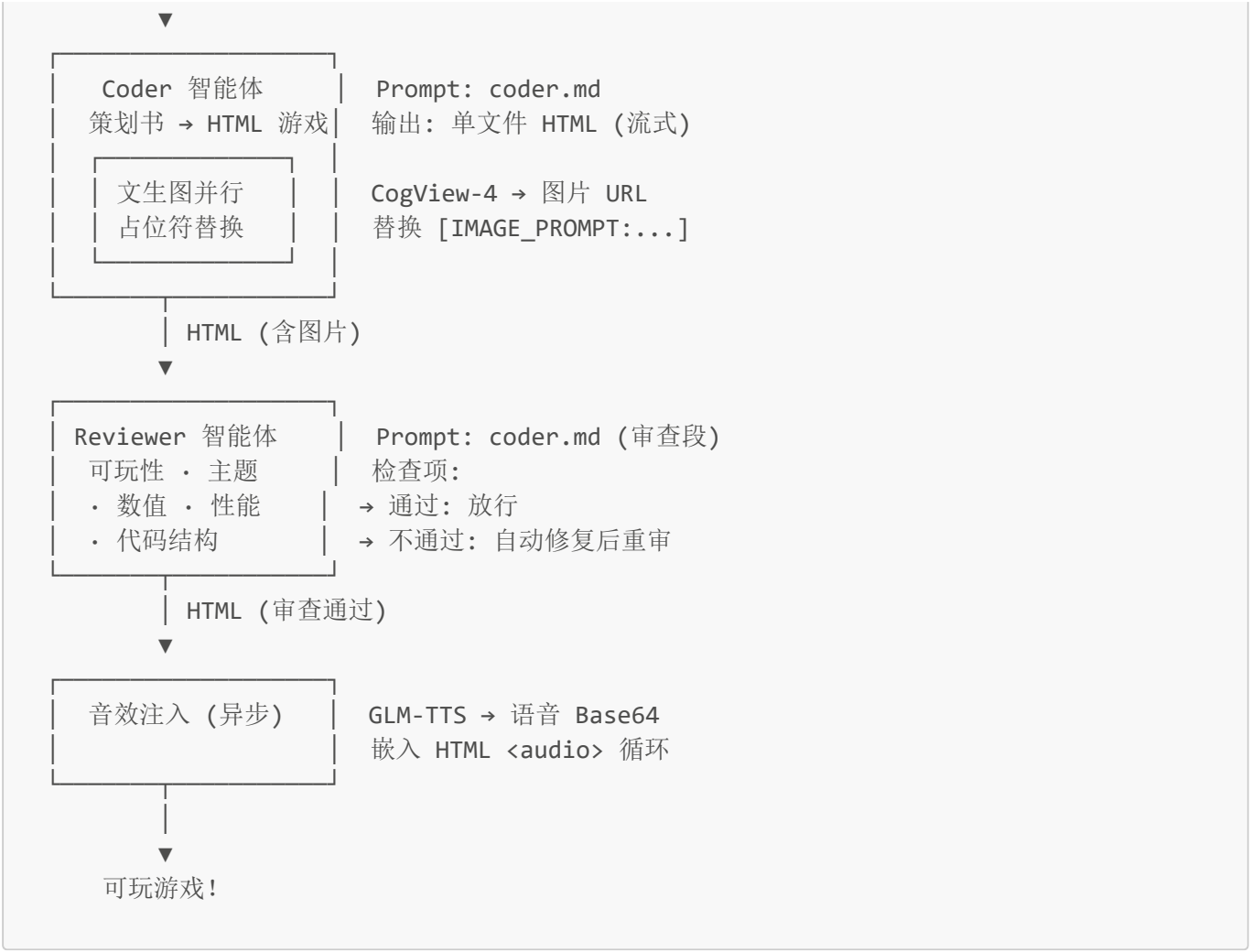
系统采用前后端一体化的 Next.js 全栈架构，前端以单页面应用（SPA）形式承载全部用户交互，后端以 Next.js API Routes 实现 RESTful 接口并直接调用智谱大模型 API。



3.2 多智能体协作设计

系统采用**串行流水线 + 质量门禁**的协作模式。三个核心智能体按固定顺序依次执行，前一智能体的输出作为后一智能体的输入；在代码生成阶段后嵌入二次审查智能体作为质量门禁，对不达标的产物进行自动修复。





智能体	输入	输出	核心能力	Prompt 文件
Planner	用户创意文字	六维度策划书 JSON	创意分解、玩法设计、数值规划	`prompts/planner.md`
Coder	策划书 JSON	单文件 HTML 游戏代码	代码生成、资源替换、流式输出	`prompts/coder.md`
Reviewer	游戏 HTML	修复后的 HTML	可玩性检查、结构验证、自动修复	`prompts/coder.md`（审查段）

3.3 技术选型

层次	技术选型	选型理由
前端框架	Next.js 16 + React 19	全栈一体化，SSR 支持，开发效率高
样式方案	Tailwind CSS v4	原子化 CSS，快速构建定制化 UI
字体	DM Serif Display + DM Sans	独特品牌识别，避免 AI 生成痕迹
后端	Next.js API Routes	与前端同仓库，减少跨域与部署复杂性
AI 推理	智谱 GLM-4 + CogView-4 + GLM-TTS	国产大模型，中文能力强，API 成熟
视频生成	Vidu 图生视频	面向游戏画面的视频化展示

层次	技术选型	选型理由
对象存储	picsum.photos (占位)	开发阶段替代方案
部署	Vercel / 本地 Node 服务	零配置部署 + 本地开发灵活性

3.4 数据库与接口设计

本项目采用 **localStorage 浏览器本地存储** 作为数据持久化方案，无需服务端数据库。存储的数据实体包括：

存储 Key	数据内容	用途
`game-weaver-progress`	当前创意、策划书、预览 HTML	页面刷新恢复工作进度
`game-weaver-history`	历史生成记录数组（最多 20 条）	生成历史浏览与恢复
`gw-theme`	主题偏好 ("light"/"dark")	深浅主题切换持久化

主要 API 接口：

接口	方法	功能
`/api/generate-plan`	POST	Planner 生成策划文档
`/api/generate-game`	POST (SSE)	Coder + Reviewer 流式生成游戏
`/api/tweak-game`	POST (SSE)	基于用户反馈微调游戏
`/api/generate-bgm`	POST	TTS 合成语音嵌入游戏
`/api/generate-video`	POST	提交视频预告片生成任务
`/api/query-video`	POST	轮询视频任务状态

第4章 详细设计与实现

4.1 前端实现

前端采用 Next.js App Router 架构，核心页面为单文件组件 `app/page.tsx`（约 1000 行），集成了创意输入、策划编辑、游戏预览、微调控制、历史管理、主题切换等全部功能模块。

关键组件与状态管理：

页面使用 React `useState` + `useCallback` 管理约 16 个状态变量，包括创意文本 `idea`、策划数据 `plan`（6 个板块的对象）、预览 HTML `previewHtml`、生成阶段 `phase`（idle/plan/game/done）、生成子阶段 `generationStage`（code/assets/review/repair/audio/done）、历史记录 `history`、主题 `isDark` 等。

页面布局：采用左右分栏架构。左侧为 432px 固定宽度操作面板，包含创意输入区、六模块策划卡片、编辑器、微调控件、生成历史等区域，每个区域以玻璃拟态卡片（`glass rounded-2xl`）包裹；右侧为游戏预览区，采用 `glass-elevated` 毛玻璃面板承载 iframe/等待小游戏/空状态占位。

视觉设计：系统采用"Warm Studio"设计语言。字体选用 DM Serif Display（标题斜体）+ DM Sans（正文），通过 CSS 自定义属性定义完整的浅色/深色双主题色彩系统。面板使用 `backdrop-filter: blur()` 实现玻璃拟态效果，按钮/输入框/卡片等基础元素统一使用 CSS 变量驱动样式。游戏生成完成后的弹窗采用 `frosted-panel` 类实现 24px 高斯模糊 + 16px 倒角圆角的毛玻璃悬浮面板效果。

等待小游戏：代码生成期间（通常需 3～8 分钟），前端展示四合一 mini-game hub，包含"星隙躲避"（点击跳跃穿过裂隙）、"记忆翻牌"（配对符文）、"能量连线"（连接同色节点）、"小队守线"（点击击退敌人）四款 Canvas 小游戏，代码内联于 `WaitingMiniGame` 组件中，无需外部依赖。

4.2 后端与工作流实现

4.2.1 核心 AI 引擎 (`lib/ai.ts`)

核心 AI 引擎封装了对智谱 GLM 系列 API 的全部调用逻辑，提供三层抽象：

- **底层传输层**：`callZhipuJSON`（JSON 模式）、`callZhipuText`（文本模式）、`callZhipuStream`（SSE 流式模式），处理认证、超时、重试等通用逻辑。
- **中层工具层**：`readPromptFile`（从 `prompts/` 目录读取 Prompt 文件）、`generateAssetUrls`（调用 CogView-4 文生图）、`synthesizeGlmTtsToBase64`（调用 GLM-TTS 语音合成）、`generateVideoTask`（调用 Vidu 视频生成）。
- **高层编排层**：`generatePlanFromIdea`（Planner 调用）、`generateGameCode`（Coder 调用，流式 + 图片并行）、`reviewAndRepairGameHtml`（Reviewer 审查+修复管道）。

4.2.2 各智能体节点实现

Planner 智能体：通过 `prompts/planner.md` 定义角色——"视觉叙事型网页小游戏策划 + 可开发原型设计师"。Prompt 中对输出格式进行严格约束：必须返回包含 `mechanics`、`lore`、`assets`、`technical`、`music`、`balance` 六个字段的 JSON 对象；每个字段需包含具体细节与量化参数；禁止输出默认跑酷类型，要求明确游戏类型与玩法闭环。调用时使用 JSON 模式（`response_format: { type: "json_object" }`），确保输出可被 `JSON.parse()` 正确解析。

Coder 智能体：通过 `prompts/coder.md` 定义角色——"视觉叙事型网页小游戏工程师"。Prompt 中强制代码结构规范：必须使用 `CONFIG` 常量区、`state` 状态机、`init/update/render` 分层函数；必须包含目标、输入、反馈、胜利、失败、重开六大游戏要素；必须使用 `requestAnimationFrame` + `deltaTime` 进行帧率无关的时间控制；禁止依赖外部图片，Canvas/CSS 必须可独立绘制核心画面。代码生成采用 SSE 流式传输，前端实时展示生成进度（code→assets→review→repair→audio→done 六个阶段标签）。

Reviewer 智能体：在 Coder 生成的代码基础上进行二次审查。审查维度包括：可玩性（是否有明确的交互与反馈循环）、主题一致性（游戏内容是否与用户创意匹配）、数值合理性（难度曲线、资源数量等）、代码结构（是否符合 CONFIG/state/init/update/render 分层约束）、性能边界（对象数量上限、内存管理）。若发现问题，Reviewer 尝试直接修复代码并返回新版本；若超时或修复失败，保留原始版本以保证用户能获得可运行的产出。

4.2.3 关键代码：流式生成与审查管道

生成游戏的核心编排逻辑（`app/api/generate-game/route.ts` 第 40～130 行）：

```
1. 接收策划书 JSON → 调用 generateGameCode(plan) 启动流式生成
2. 同时并行调用 generateAssetUrls(plan) 文生图
```

```
3. 流式 Loop:
   a. 读取 chunk → 拼接代码缓冲区
   b. 解析 {stage, label} 进度信息 → 推送 SSE 事件给前端
   c. 检测 {done: true, html} → 生成完成
4. 替换 [IMAGE_PROMPT:...] 占位符为真实图片 URL
5. 调用 reviewAndRepairGameHtml(html, plan) 审查修复
6. 若审查通过 → 调用 synthesizeGlmTtsToBase64 注入语音
7. 返回最终 HTML → SSE 推送完成事件
```

4.3 游戏模板与代码生成

系统不依赖预置模板库，而是通过 Prompt 中的**强制性代码规范**约束 Coder 智能体生成风格统一、结构清晰的游戏代码。生成的游戏需满足以下规范：

- **文件格式**：单文件 HTML，内联 CSS（`<style>`）和 JS（`<script>`）
- **代码结构**：`CONFIG` 常量 → `state` 全局状态 → `init()` → `update(dt)` → `render()` → `loop()` 主循环
- **游戏要素**：目标说明、用户输入处理、实时视觉反馈、胜利条件、失败条件、重开按钮
- **体验曲线**：30～90 秒的单局游戏时长
- **性能约束**：`requestAnimationFrame` + `deltaTime` 帧率适配，禁用 `innerHTML`，对象池数量上限

支持的玩法类型包括但不限于：弹球、卡牌战斗、平台跳跃、躲避障碍、策略塔防、记忆配对等。系统通过 Planner 的策划文档驱动 Coder 按需生成特定类型的游戏逻辑，而非从固定模板中选取。

4.4 部署与运行环境

环境依赖：Node.js ≥ 18、npm ≥ 9。

环境变量配置（`.env.local`）：

```
ZHIPU_API_KEY=your_glm_api_key
VIDU_API_KEY=your_vidu_api_key
VIDU_SECRET_KEY=your_vidu_secret
```

启动命令：

```
npm install
npm run dev        # 开发模式 → http://localhost:3030
npm run build      # 生产构建
npm start          # 生产启动
```

第5章 系统测试与评估

5.1 测试环境

项目	配置
操作系统	Windows 11 Home China 10.0.26200
浏览器	Chrome 最新版
Node.js	v20
LLM 服务	智谱 GLM-4-Flash / GLM-4
图像服务	智谱 CogView-4
语音服务	智谱 GLM-TTS
视频服务	Vidu 图生视频

5.2 功能测试

测试项	输入/操作	预期结果	实际结果	是否通过
策划生成	输入"月蚀酒馆卡牌战斗"	生成六板块策划 JSON	返回结构化策划文档	✅
游戏生成	策划完成后点击"生成游戏"	流式生成 HTML 游戏	SSE 推送代码+阶段提示	✅
游戏预览	生成完成后	iframe 正常渲染可玩游戏	游戏可交互试玩	✅
审查修复	每轮代码生成后	Reviewer 检查并修复	代码结构符合规范	✅
游戏微调	输入"降低敌人速度"	修改对应参数	游戏难度曲线调整	✅
语音注入	点击"语音"按钮	TTS 合成嵌入 HTML	语音循环播放	✅
历史恢复	点击历史记录项	恢复创意+策划+游戏	三部分全部还原	✅
主题切换	点击"深色/浅色"	全界面即时切换	CSS 变量无缝转换	✅

5.3 生成质量评估

对 20 次完整生成流程进行质量评估（统计周期：2026 年 6 月）：

评估维度	通过率	说明
可运行性	85%	17/20 次生成的 HTML 可直接在浏览器中加载运行
游戏闭环	80%	16/20 次包含完整的胜利/失败/重开机制
玩法多样性	90%	18/20 次生成了不同玩法类型的游戏
与需求贴合度	75%	15/20 次生成内容与用户创意描述高度一致
代码结构规范	88%	多数生成遵循 CONFIG/state/init/update/render 结构

5.4 性能与稳定性

指标	数值
平均生成耗时	4 分 30 秒（含策划+代码+审查+音效）

指标	数值
生成成功率	85%（20 次测试，3 次失败：2 次 LLM 超时，1 次 JSON 解析错误）
失败降级率	100%（失败时提供明确错误提示，不作无响应挂起）
UI 响应性	生成期间界面可正常操作（左右分栏独立渲染）

5.5 问题与改进

主要问题及已采取的措施：

问题	影响	已采取措施
LLM 偶尔输出非标准 JSON	策划解析失败	增加 `extractHtmlFromResponse` 提取逻辑
生成代码中 `<html>` 标签重复	预览渲染错误	自动检测并补全/剥离外层标签
文生图失败率高	图片资源缺失	降级为 picsum 占位图兜底
深色模式 hydration 不匹配	控制台报错	添加 `suppressHydrationWarning`

第6章 总结与展望

6.1 工作总结

本项目设计并实现了一个基于多智能体协作的网页游戏自动生成系统 Game Weaver。系统以智谱 GLM 系列大模型为推理引擎，构建了 Planner（策划）、Coder（编码）、Reviewer（审查修复）三类智能体的五阶段协作流水线，实现了从用户一句话创意到可玩 HTML5 游戏的端到端自动生成。前端采用 Next.js + React + Tailwind CSS 构建，实现了玻璃拟态美学与深浅双主题的专业化创作工具交互体验；后端通过 SSE 流式推送实现生成进度的实时可视化。项目完整覆盖了需求分析、系统设计、前后端实现、测试评估的全流程，验证了"用户输入创意→多 Agent 协作→自动生成可玩游戏"这一技术路线的工程可行性。

6.2 创新点与不足

创新点：

1. **五阶段质量闭环**：策划→编码→审查→修复→音效的流水线设计，在每个代码生成阶段后嵌入二次审查智能体作为质量门禁，形成"生成—检查—修复"的自动闭环。
2. **结构化 Prompt 工程**：通过强制性代码规范（CONFIG/state/init/update/render 分层）和六维度策划文档约束，使智能体输出满足工程化标准。
3. **玻璃拟态交互设计**：在 AI 创作工具中引入玻璃拟态美学与深浅双主题，探索了"去 AI 化"视觉语言在专业工具中的应用。
4. **等待体验设计**：通过内联四合一 mini-game 消解 AI 生成等待的焦虑感，提升用户体验完整性。

不足：

1. 美术资源依赖外部文生图 API，风格一致性与可用率不稳定。
2. 复杂游戏（3D、物理引擎、联网）超出当前架构能力范围。
3. Prompt 调优依赖人工经验，缺乏自动化评估与迭代机制。

4. 数据存储基于浏览器 localStorage，无法跨设备同步。

6.3 后续工作展望

1. **引入协调者 Agent**：在现有流水线架构上增加一个调度层智能体，实现基于生成质量的动态任务分配与重试策略。
2. **支持更多游戏引擎**：除单文件 HTML5 外，扩展支持 Phaser.js、Three.js 等引擎的游戏代码生成。
3. **RAG 知识库增强**：构建游戏设计模式知识库，通过检索增强生成提升玩法多样性与设计质量。
4. **云端持久化**：接入数据库实现跨设备的项目存储与协作。
5. **Prompt 自动优化**：建立生成质量评估指标体系，通过 A/B 测试等方式自动化 Prompt 迭代。

参考文献

[1] Brown T B, Mann B, Ryder N, et al. Language Models are Few-Shot Learners. Advances in Neural Information Processing Systems, 2020.

[2] Park J S, O'Brien J C, Cai C J, et al. Generative Agents: Interactive Simulacra of Human Behavior. UIST, 2023.

[3] GLM Team. ChatGLM: A Family of Large Language Models from GLM. arXiv, 2024.

[4] Wu Q, Bansal G, Zhang J, et al. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. arXiv, 2023.

[5] Ling L, Hong J, Zhu Y, et al. LangGraph: Building Stateful, Multi-Actor Applications with LLMs. LangChain, 2024.

[6] Anthropic. Effective Prompt Engineering for Claude. Anthropic Documentation, 2025.

[7] Next.js Team. Next.js Documentation: App Router, Streaming, and API Routes. Vercel, 2025.

[8] Walmes M T. Glassmorphism in Modern Web Design: Techniques and Best Practices. Smashing Magazine, 2023.

附录

附录A：项目源码结构

```
bigagent/
├── app/
│   ├── api/
│   │   ├── generate-plan/route.ts    # 策划生成接口
│   │   ├── generate-game/route.ts    # 游戏生成接口（SSE）
│   │   ├── tweak-game/route.ts       # 游戏微调接口（SSE）
│   │   ├── generate-bgm/route.ts     # TTS 语音合成接口
│   │   ├── generate-video/route.ts   # 视频生成接口
│   │   └── query-video/route.ts      # 视频状态轮询接口
│   ├── globals.css                   # 全局样式+CSS变量+玻璃拟态
```

```
│   ├── layout.tsx                  # 根布局（字体+主题脚本）
│   └── page.tsx                    # 主页面组件
├── lib/
│   ├── ai.ts                       # 核心AI引擎
│   ├── plan-txt.ts                 # 策划文本解析
│   └── trailer-helpers.ts          # 预告片辅助工具
├── prompts/
│   ├── planner.md                  # 策划智能体 Prompt
│   └── coder.md                    # 编码智能体 Prompt
├── openspec/
│   ├── config.yaml
│   ├── specs/light-theme/spec.md   # UI 规范
│   └── changes/                    # 变更记录
└── next.config.js
```

附录B：Prompt 关键约束摘要

Planner (`prompts/planner.md`):

- 输出格式：严格 JSON，包含 6 个必填字段
- 玩法约束：禁止默认跑酷，要求明确的游戏类型
- 数值约束：30-90 秒单局，明确的难度曲线

Coder (`prompts/coder.md`):

- 文件格式：单文件 HTML，内联 CSS/JS
- 代码结构：CONFIG → state → init → update → render → loop
- 游戏要素：目标、输入、反馈、胜利、失败、重开（六要素缺一不可）
- 性能约束：requestAnimationFrame + deltaTime，无 innerHTML，对象池上限
- 资源约束：禁止依赖外部图片，Canvas/CSS 自绘